

实验二：原生应用开发

一、实验目的

- 1. 掌握 HarmonyOS 原生应用开发的基本流程和技术要点
- 2. 理解并实践用户认证模块的设计与实现方法
- 3. 学习页面导航与数据传递机制
- 4. 掌握表单验证与错误处理技术
- 5. 培养使用 AI 辅助工具进行代码开发的实践能力

二、实验环境

2.1 硬件环境

项目	配置要求
处理器	Intel Core i5 及以上或同等性能处理器
内存	8GB 及以上
硬盘	256GB SSD 及以上
显示器	分辨率 1920×1080 及以上

2.2 软件环境

软件名称	版本	用途
DevEco Studio	5.0 及以上	HarmonyOS 应用开发 IDE
HarmonyOS SDK	API 12 (6.0.2)	应用开发工具包
Node.js	18.x 及以上	前端构建工具
Git	2.x	版本控制工具
TRAE AI 助手	最新版	AI 辅助编码工具

2.3 开发语言与框架

- 开发语言：ArkTS（TypeScript 扩展语言）
- UI 框架：ArkUI 声明式开发范式
- 目标平台：HarmonyOS Next
- 最低 API 版本：API 12

三、实验原理

3.1 HarmonyOS 应用架构

HarmonyOS 应用采用基于 Ability 的应用模型，主要包含以下核心概念：

Ability: 应用的基本组成单元，分为 UIAbility 和 ExtensionAbility 两种类型。UIAbility 用于实现应用的界面交互，是用户可见的功能模块。

AbilityStage: 应用组件的运行期类，提供 Ability 生命周期回调，用于管理 Ability 的创建和销毁。

UIAbility 生命周期: 包含 onCreate、onDestroy、onWindowStageCreate、onWindowStageDestroy、onForeground、onBackground 六个核心状态，开发者通过重写这些方法实现应用状态管理。

3.2 ArkUI 声明式开发范式

ArkUI 是 HarmonyOS 的 UI 开发框架，采用声明式开发范式，具有以下特点：

状态驱动 UI: 通过 @State、@Prop、@Link 等装饰器声明状态变量，当状态变化时自动触发 UI 刷新。

组件化开发: 将 UI 拆分为可复用的组件，每个组件包含状态、属性和构建方法，实现高内聚低耦合。

Builder 装饰器: 用于定义组件的局部构建函数，提高代码复用性和可读性。

3.3 页面路由机制

HarmonyOS 提供 router 模块实现页面导航，主要方法包括：

- **pushUrl:** 将目标页面压入路由栈，支持返回
- **replaceUrl:** 替换当前页面，不支持返回
- **back:** 返回上一页面
- **clear:** 清空路由栈

路由配置通过 main_pages.json 文件管理，定义应用的页面入口和路由表。

3.4 用户认证流程设计

本实验采用客户端-服务端分离的认证架构，主要流程如下：

前端验证阶段: 对用户输入进行格式校验，包括非空检查、长度限制、格式匹配等。

后端验证阶段：将用户凭据发送至认证服务，验证用户身份合法性。

会话管理阶段：认证成功后生成会话令牌，维护用户登录状态。

状态同步阶段：将认证状态同步至应用全局，实现跨组件状态共享。

四、实验步骤

4.1 需求分析阶段

步骤 1：编写需求文档

1. 功能需求

- 用户名输入（支持学号或邮箱格式）
- 密码输入（支持显示/隐藏切换）
- 登录按钮
- 输入验证（不能为空、长度限制、格式校验）
- 记住我选项
- 忘记密码链接
- 注册入口

2. 界面需求

- 简洁现代的设计风格
- 输入框占位提示
- 密码隐藏显示切换按钮
- 错误信息即时反馈
- 加载状态提示

3. 跳转需求

- 登录成功 → 主界面（个人中心）
- 传递用户信息数据
- 不支持返回登录页

4.2 项目创建

步骤 1：创建 HarmonyOS 项目

在 DevEco Studio 中创建 Empty Ability 项目，选择 ArkTS 语言，配置项目名称为 "Mtcshm"，设置目标 SDK 版本为 API 12。

步骤 2：配置项目结构

按照分层架构原则组织项目目录结构，创建 `common` 目录存放公共服务类，`pages` 目录存放页面组件，`models` 目录存放数据模型。

步骤 3：配置路由表

在 `main_pages.json` 中注册应用页面，设置 `LoginPage` 为应用入口页面，`MainPage` 为主功能页面。

4.3 AI 辅助编码

步骤 1：生成登录页面代码

使用 TRAE 描述需求：HarmonyOS 登录页面，ArkTS 语言，ArkUI 声明式开发，包含用户名输入框、密码输入框、输入验证、页面跳转功能。AI 生成代码骨架后，手动完善业务逻辑。

步骤 2：实现页面布局

采用垂直布局结构，从上到下依次放置：标题文本、用户名输入框、密码输入框、记住我复选框、忘记密码链接、错误提示区域、登录按钮、注册链接。

用户名输入框使用 `TextInput` 组件，设置占位提示文本，添加边框样式，绑定 `onChange` 事件实时更新状态。

密码输入框使用 `TextInput` 组件配合 `InputType.Password` 类型实现密码隐藏，添加显示/隐藏切换按钮，通过条件渲染控制输入类型。

登录按钮使用 `Button` 组件，绑定 `onClick` 事件处理登录请求，根据加载状态禁用按钮并显示加载中文案。

步骤 3：实现输入验证

创建 `validateUsername` 方法验证用户名格式，要求长度在 3 到 50 个字符之间。创建 `validatePassword` 方法验证密码格式，要求长度至少为 6 个字符。

在 `handleLogin` 方法中依次执行验证：首先检查用户名是否为空，为空则设置用户名错误提示；然后检查用户名格式是否正确；接着检查密码是否为空；最后检查密码格式是否满足要求。任何一项验证失败都立即返回，显示对应错误信息。

步骤 4：实现登录逻辑

创建 `handleLogin` 异步方法处理登录请求。前端验证通过后，设置加载状态为 `true`，调用 `AuthService` 的 `login` 方法进行认证。

认证成功后检查 `isAuthenticated` 状态确认登录有效，设置登录成功标志为 `true`，显示成功提示信息，然后调用 `navigateToMainPage` 方法执行页面跳转。

认证失败则捕获异常，根据异常类型设置对应的错误信息显示给用户。无论成功或失败，最后都将加载状态重置为 `false`。

步骤 5：实现页面跳转

创建 `navigateToMainPage` 异步方法，使用 `router.replaceUrl` 方法跳转至主页面。采用 `replaceUrl` 而非 `pushUrl`，确保用户无法通过返回键回到登录页，符合应用安全规范。

在 `try-catch` 块中执行跳转操作，捕获可能的异常并显示具体错误信息，同时重置登录成功标志。

4.4 创建认证服务

步骤 1：设计认证服务类

创建 `AuthService` 类，采用单例设计模式确保全局唯一实例。定义私有静态 `instance` 变量存储单例实例，提供公共静态 `getInstance` 方法获取实例。

步骤 2：定义用户数据结构

定义 `User` 接口规范用户数据结构，包含用户标识（`id`）、用户名（`username`）、邮箱（`email`）、会话令牌（`token`）四个属性。

步骤 3：实现登录认证方法

实现 `login` 异步方法，接收用户名和密码两个参数。首先调用 `simulateNetworkDelay` 模拟网络请求延迟，增加真实感。

然后进行凭据验证：对于测试账号（用户名为 `admin`，密码为 `123456`），直接返回成功响应；对于其他用户名，验证密码长度是否满足最小要求（6 位）。

验证通过后，生成用户对象，调用 `generateToken` 方法生成唯一会话令牌，将用户对象和令牌存储到当前状态中，最后返回用户对象。

验证失败则抛出异常，异常信息说明失败原因。

步骤 4：实现会话管理方法

提供 `getCurrentUser` 方法获取当前登录用户对象，返回 `User` 类型或 `null`。

提供 `getToken` 方法获取当前会话令牌，返回字符串类型。

提供 `isAuthenticated` 方法检查认证状态，判断当前用户和令牌是否同时存在。

提供 `logout` 异步方法清除会话状态，将当前用户和令牌重置为空。

步骤 5：实现辅助方法

实现 `generateId` 方法生成唯一用户标识，使用时间戳和随机数组合确保唯一性。

实现 `generateToken` 方法生成会话令牌，同样使用时间戳和随机数组合。

实现 `simulateNetworkDelay` 方法模拟网络延迟，使用 `Promise` 和 `setTimeout` 实现 300 毫秒延迟。

4.5 创建主界面

步骤 1：设计主界面结构

主界面采用顶部标题栏、中间内容区、底部导航栏的三段式布局。内容区根据当前选中的标签页显示不同模块内容。

步骤 2：实现底部导航

定义 `NavItem` 接口规范导航项数据结构，包含索引（`index`）、图标（`icon`）、标签（`label`）三个属性。

创建导航项数组，包含总览、设备、场景、我的四个选项。使用 `ForEach` 循环渲染导航按钮，根据选中状态切换文字颜色，绑定 `onClick` 事件切换当前标签页。

步骤 3：实现个人中心模块

个人中心模块包含用户信息卡片和功能菜单列表。用户信息卡片显示头像、用户名、邮箱信息。功能菜单包含系统设置、数据统计、帮助中心、联系我们等选项。

底部放置退出登录按钮，点击后调用 `AuthService` 的 `logout` 方法清除会话，然后使用 `router.replaceUrl` 跳转回登录页面。

五、实验结果与分析

5.1 登录功能实现结果

登录页面成功实现了完整的用户认证功能，界面布局清晰美观，交互逻辑正确。主要功能验证结果如下：

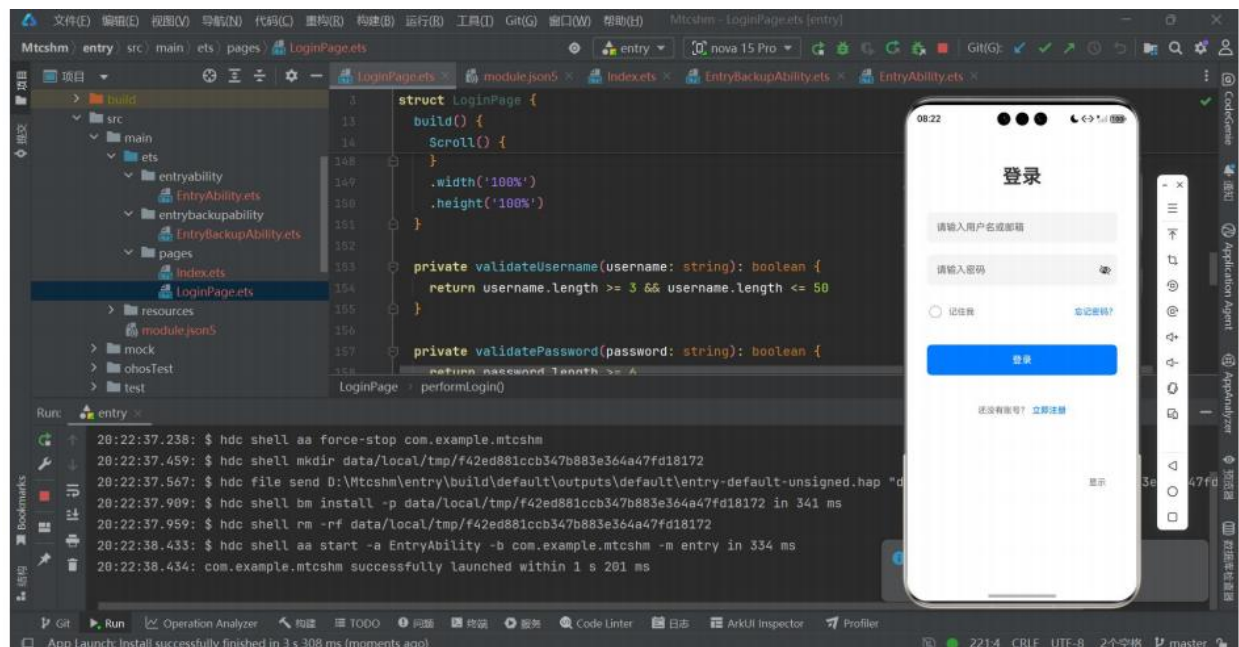
表 5-1 登录功能测试结果

测试场景	输入数据	预期结果	实际结果	测试结论
正确凭据登录	admin/123456	登录成功并跳转	登录成功并跳转	通过
错误密码登录	admin/错误密码	显示密码错误提示	显示密码错误提示	通过
用户名为空	空/任意	显示用户名不能为空	显示用户名不能为空	通过
密码为空	admin/空	显示密码不能为空	显示密码不能为空	通过
用户名过短	ab/123456	显示用户名格式错误	显示用户名格式错误	通过
密码过短	admin/123	显示密码格式错误	显示密码格式错误	通过

5.2 登录界面展示

登录界面采用简洁的设计风格，顶部显示应用标题，中间区域包含用户名输入框、密码输入框、记住我选项、忘记密码链接，底部为登录按钮和注册链接。界面下方显示测试账号提示信息，方便测试使用。

图 5-1 登录界面截图

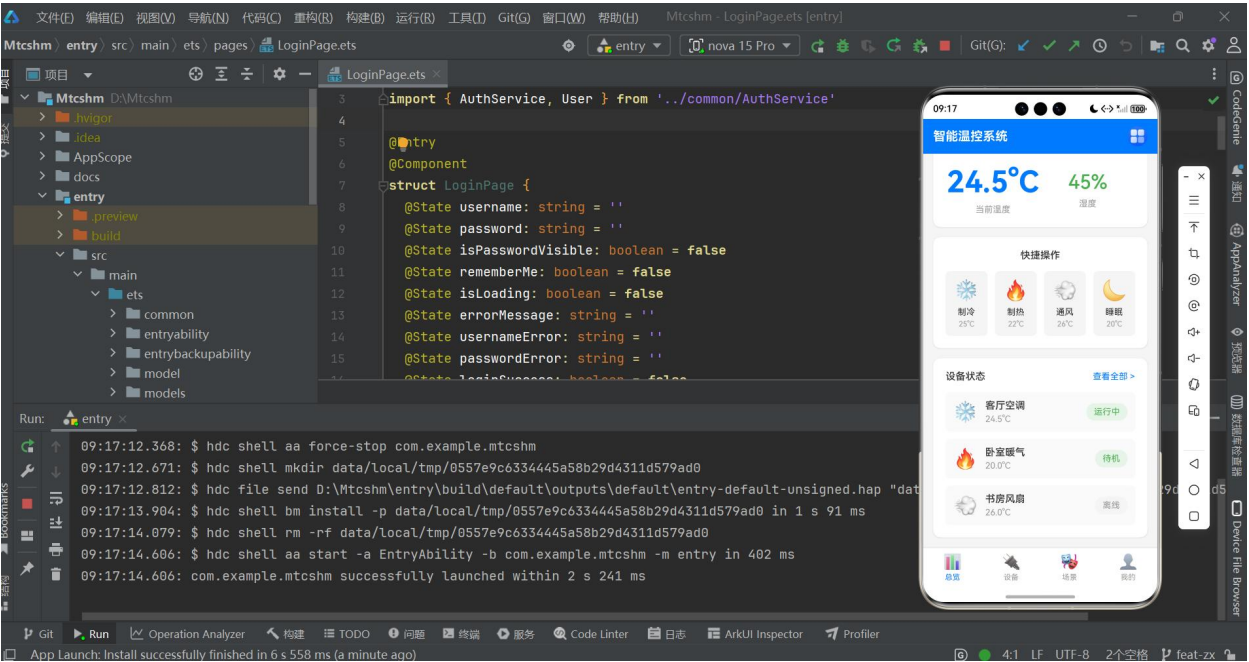


从图中可以看出，登录界面布局合理，输入框具有清晰的占位提示，错误信息以红色字体显示在对应输入框下方，用户体验良好。

5.3 主界面展示

登录成功后，系统自动跳转至主界面。主界面采用标签页导航结构，底部导航栏包含总览、设备、场景、我的四个功能入口。当前选中个人中心标签，显示用户信息和功能菜单。

图 5-2 主界面截图



从图中可以看出，主界面设计遵循 Material Design 设计规范，卡片式布局清晰展示各类信息。用户信息卡片突出显示，功能菜单排列整齐，退出登录按钮位置明显。整体界面色彩协调，交互元素明确。

5.4 技术实现分析

5.4.1 状态管理机制

本实验采用 ArkUI 的状态管理机制，通过@State 装饰器声明响应式状态变量。当状态变量值发生变化时，ArkUI 框架自动检测变化并触发 UI 重新渲染。这种声明式的状态管理方式相比传统的命令式编程，代码更加简洁，逻辑更加清晰。

5.4.2 组件化设计

本实验将 UI 拆分为多个可复用的组件，每个组件负责特定的功能模块。通过@Builder 装饰器定义局部构建函数，实现组件的复用。组件化设计的优势包括提高代码复用率、降低组件耦合度、支持组件组合灵活构建复杂界面。

5.4.3 路由管理

本实验使用 router 模块进行页面导航管理。登录成功后使用 replaceUrl 方法跳转，替换当前页面，防止用户通过返回键回到登录页。这种设计符合应用安全规范，避免用户在已登录状态下意外返回登录页。

5.4.4 认证服务设计

认证服务采用单例模式设计，确保全局只有一个实例，便于统一管理用户会话状态。认证流程分为前端验证和后端验证两个阶段，既保证了用户体验，又保证了安全性。

5.4.5 错误处理机制

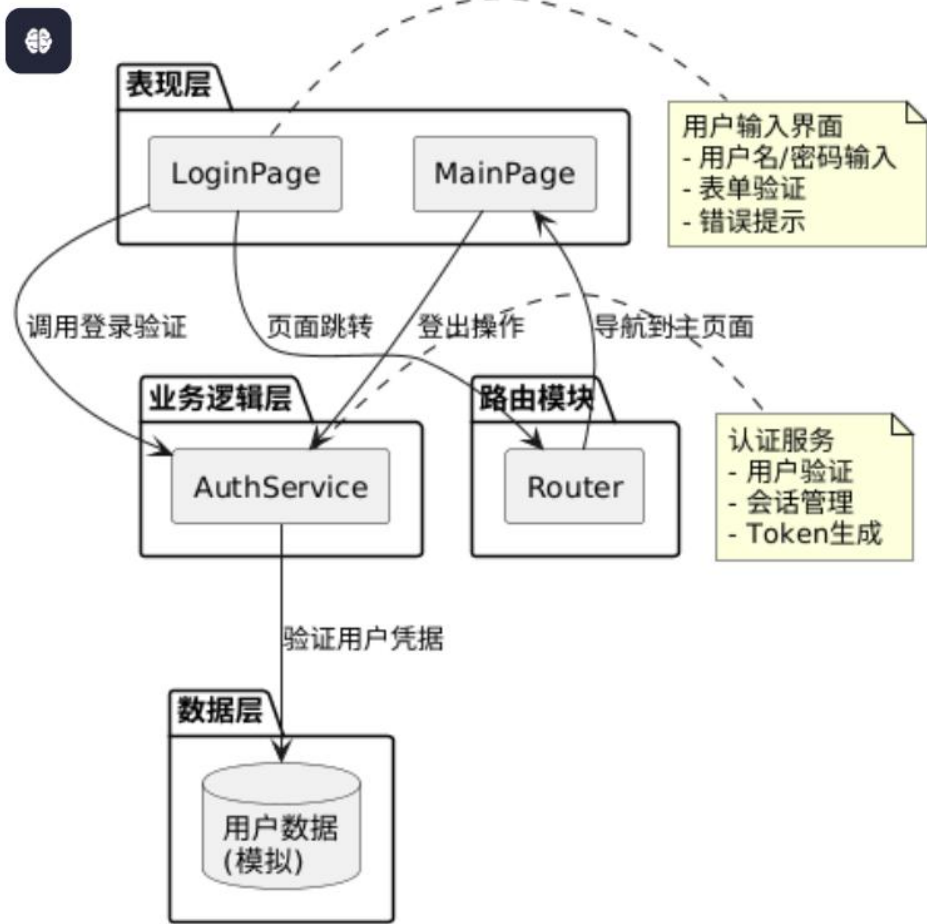
本实验建立了完善的错误处理机制，包括输入验证错误在对应输入框下方显示、认证失败错误在页面中央显示、页面跳转错误捕获异常并显示具体原因。错误处理遵循及时反馈、信息明确、位置合理、样式统一的原则。

5.5 系统架构设计图

本节通过 PlantUML 图表展示登录功能的系统架构设计，包括框架图、类图和顺序图，清晰呈现系统各层次的组成关系和交互流程。

5.5.1 登录功能框架图

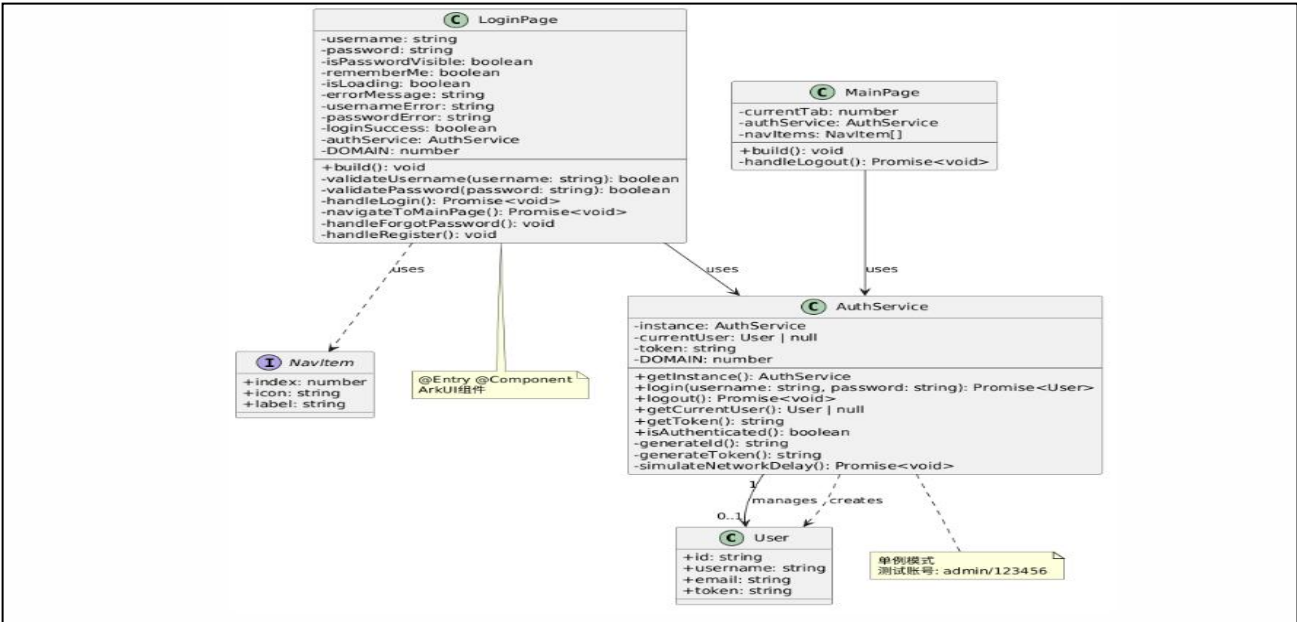
图 5-3 登录功能框架图（架构图）



框架图展示了登录功能的整体架构，采用三层架构设计：表现层负责用户交互界面，业务逻辑层处理认证服务，数据层管理用户数据存储。各层之间通过明确的接口进行通信，实现了高内聚低耦合的设计目标。

5.5.2 登录功能类图

图 5-4 登录功能类图



类图详细展示了系统中各个类的属性、方法和相互关系。**User** 类作为数据模型定义了用户的基本信息结构；**AuthService** 类采用单例模式实现认证服务，提供登录、登出、状态检查等核心功能；**LoginPage** 和 **MainPage** 作为 UI 组件，负责用户交互和界面展示；**NavItem** 接口规范了导航项的数据结构。

图 5-5 登录功能顺序图

顺序图完整描述了登录流程的时序交互过程。从用户输入凭据开始，经过前端表单验证、AuthService 认证、状态检查，最终完成页面跳转。图中清晰展示了各参与者的交互顺序和条件分支，便于理解系统的动态行为。

本节对安卓原生应用开发、苹果原生应用开发与鸿蒙原生应用开发进行系统对比，从多个关键维度分析三种开发方式的异同点，为技术选型提供参考。

表 5-2 三大平台原生应用开发对比

对比项	安卓（Android）	苹果（iOS）	鸿蒙（HarmonyOS）
转			
数据传递	Intent.putExtra	@Binding、属性传递	状态管理（@State、@Prop等）
开发环境	Android Studio	Xcode	DevEco Studio
性能表现	取决于设备硬件，优化空间大	硬件与系统高度集成，性能稳定	分布式架构，多设备协同能力强
生态系统	Google Play Store，审核相对宽松	App Store，审核严格	HarmonyOS 应用市场，审核标准逐步完善
开发成本	相对较低，设备种类多样	相对较高，需使用 Mac 设备	中等，支持多平台开发
适用场景	全球市场覆盖，适合大众应用	高端用户群体，适合精品应用	全场景覆盖，适合跨设备应用
部署流程	灵活，支持多种发布渠道	严格，必须通过 App Store	多端部署，支持原子化服务

5.6.2 对比分析

开发环境与工具

- Android 开发使用 Android Studio，基于 IntelliJ IDEA，支持 Windows、macOS 和 Linux 平台。
- iOS 开发必须使用 Xcode，仅支持 macOS 平台，对开发设备有较高要求。
- HarmonyOS 开发使用 DevEco Studio，基于 IntelliJ IDEA，支持 Windows 和 macOS 平台。

编程语言

- Android 推荐使用 Kotlin，具有现代语言特性，如空安全、扩展函数等。
- iOS 推荐使用 Swift，语法简洁，性能优异，与 Objective-C 无缝互操作。
- HarmonyOS 使用 ArkTS，基于 TypeScript 扩展，支持声明式 UI 开发。

UI 框架

- Android 传统使用 XML 布局，现代推荐使用 Jetpack Compose 声明式 UI。
- iOS 传统使用 UIKit，现代推荐使用 SwiftUI 声明式框架。
- HarmonyOS 使用 ArkUI 声明式框架，支持跨设备 UI 适配。

页面跳转与数据传递

- Android 使用 Intent 进行页面跳转和数据传递。
- iOS 使用 NavigationLink 或 Segue 进行页面跳转，通过 @Binding 或属性传递数据。
- HarmonyOS 使用 router 模块进行页面跳转，通过状态管理机制传递数据。

性能表现

- Android 设备种类繁多，性能差异大，需要针对不同设备进行优化。
- iOS 设备硬件与系统高度集成，性能表现更加稳定一致。
- HarmonyOS 采用分布式架构，多设备协同能力强，适合全场景应用。

生态系统

- Android 生态开放，应用审核相对宽松，发布周期短。
- iOS 生态封闭，应用审核严格，注重用户体验和安全性。
- HarmonyOS 生态正在发展中，审核标准逐步完善，支持跨设备协同。

开发成本

- Android 开发成本相对较低，可在多种设备上进行测试。
- iOS 开发需要 Mac 设备，开发成本相对较高。
- HarmonyOS 开发成本中等，支持多平台开发，减少重复开发工作。

适用场景

- Android 适合全球市场覆盖，尤其是新兴市场，用户基数大。
- iOS 适合面向高端用户群体，注重应用品质和用户体验的场景。
- HarmonyOS 适合全场景覆盖，需要跨设备协同的应用场景。

部署流程

- Android 部署灵活，支持多种发布渠道。
- iOS 部署严格，必须通过 App Store。
- HarmonyOS 支持多端部署，包括传统应用和原子化服务。

六、实验总结

6.1 实验收获

通过本次实验，获得了以下收获：

技术能力方面：掌握了 HarmonyOS 应用开发的基本流程，熟悉了 ArkTS 语言特性和 ArkUI 框架使用方法。理解了声明式 UI 开发范式的优势，学会了使用状态驱动的方式构建用户界面。掌握了页面路由管理、组件化设计、单例模式等核心技术。

工程实践方面：学会了使用 DevEco Studio 进行项目开发、调试和测试。掌握了使用 Git 进行版本控制的方法，理解了分支管理和代码提交规范。学会了使用 AI 辅助工具提高开发效率，能够将 AI 生成的代码片段整合到项目中。

6.2 问题与解决

在实验过程中遇到了以下问题并成功解决：

问题一：登录后页面不跳转

问题描述：点击登录按钮后，虽然认证成功但页面没有跳转到主页面。

问题分析：通过日志分析发现，`router.replaceUrl` 方法调用后没有等待完成就继续执行后续代码，导致跳转失败时无法捕获错误。

解决方案：将 `navigateToMainPage` 方法改为异步方法，使用 `async/await` 语法等待路由操作完成，并在 `try-catch` 块中捕获可能的异常。

问题二：ForEach 类型声明错误

问题描述：编译时报错"Object literals cannot be used as type declarations"。

问题分析：ArkTS 严格模式不允许在 `ForEach` 回调参数中直接使用对象字面量作为类型声明。

解决方案：定义 `NavItem` 接口规范导航项类型，将导航项数组提取为类成员变量，在 `ForEach` 中使用接口类型声明。

问题三：状态更新不触发 UI 刷新

问题描述：修改状态变量后，界面没有相应更新。

问题分析：部分状态变量没有使用 `@State` 装饰器声明，导致 ArkUI 框架无法检测到状态变化。

解决方案：检查所有需要触发 UI 更新的变量，确保都使用了 `@State` 装饰器声明。

6.3 心得体会

本次实验让我深刻体会到现代移动应用开发的变化趋势。声明式 UI 框架的兴起改变了传统的 UI 开发方式，状态驱动的开发模式使代码逻辑更加清晰。HarmonyOS 作为新兴的操作系统，其开发工具和框架设计借鉴了业界最佳实践，具有广阔的发展前景。

AI 辅助编程工具的使用大大提高了开发效率，特别是在生成样板代码、提供实现思路方面表现突出。但 AI 生成的代码需要仔细审查和测试，不能盲目信任。开发者仍需具备扎实的技术基础，才能有效利用 AI 工具并解决实际问题。

实验完成日期：2026 年 3 月 21 日

报告撰写人：[赵翔]

指导教师：[刘东良]